МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
ГОМЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ИМЕНИ П.О.СУХОГО

Факультет автоматизированных и информационных систем

Кафедра «Информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №5

по дисциплине: «Интеграция больших языковых моделей в Android-приложение»

на тему: **«Программирование мобильных информационных систем»**

Выполнил: студент группы ИТП-31
Почивалов Д.С.
Проверил: ассистени
Шаблинский

Гомель 2026

ЛАБОРАТОРНАЯ РАБОТА №5

Интеграция большой языковой модели в *Android*-приложение для смысловой ревизии коллекции дисконтных карт

Цель работы: освоить локальный запуск большой языковой модели *Qwen-2.5-1.5B-Instruct* в формате *GGUF* на *Android*-устройстве, встроить генерацию в приложение на *Kotlin* без обращения к сети, реализовать проверку совместимости устройства, обработку состояний загрузки, успеха и ошибки через *StateFlow*, а также кэширование результата анализа в *Room.*

Краткие теоретические сведения

Большая языковая модель, далее *LLM*, представляет собой нейронную сеть трансформерного типа, обученную предсказывать следующий токен текста. В мобильном приложении такая модель применяется для локальной обработки естественного языка: классификации, извлечения структурированных данных, формирования кратких пояснений и проверки смыслового соответствия записей. Ключевое преимущество локального запуска состоит в том, что пользовательские данные не покидают устройство, а работа функции не зависит от доступности сети и внешних программных интерфейсов.

Для выполнения вычислений используется библиотека *llama.cpp*, реализующая эффективный вывод моделей семейства *LLaMA* на центральном процессоре. Модели распространяются в формате *GGUF*, который содержит веса, метаданные и описание токенизатора в одном файле. Формат поддерживает различные схемы квантования. В работе выбрана схема *Q4_K_M*: она уменьшает размер весов примерно до четырёх с половиной бит на параметр и обеспечивает компромисс между качеством ответа, объёмом файла и скоростью вычислений.

Модель *Qwen-2.5-1.5B-Instruct* содержит около 1,5 млрд параметров и дообучена следовать инструкциям. Для диалогового формата используется разметка *ChatML* с маркерами *im_start* и *im_end*, ролями *system, user* и *assistant.* Инструкция задаёт правила анализа, пользовательское сообщение содержит перечень категорий и карт, а ответ ассистента формируется в заданном формате *JSON.* Такая схема позволяет отделить постановку задачи от данных и упростить автоматическую проверку ответа.

На *Android* библиотека *llama.cpp* подключается через *JNI* и поставляется в виде нативных библиотек для 64-битных архитектур *arm64-v8a* и *x86_64.* Модель загружается в оперативную память и занимает около 1,1 ГБ, поэтому устройство должно располагать достаточным объёмом ОЗУ. Загрузка модели выполняется отдельно от генерации и может занимать несколько секунд, а сама генерация на мобильном процессоре существенно медленнее, чем на серверном оборудовании.

Архитектура приложения строится по слоям. *Repository* скрывает детали работы *JNI* и предоставляет единый интерфейс движка. *UseCase* описывает сценарий анализа коллекции: получает карты и категории, проверяет кэш,

формирует промпт, вызывает модель, проверяет ответ и сохраняет результат. *ViewModel* хранит состояние экрана в *StateFlow* и не содержит кода работы с нативными библиотеками. *Room* используется для кэширования последнего результата, чтобы повторное открытие экрана не требовало повторной генерации.

Вариант задания предусматривает смысловую ревизию всей коллекции дисконтных карт. Модель должна определить карты, которые находятся не в своей категории, предложить правильную категорию и объяснить причину перемещения. Ответ возвращается в структурированном виде, проверяется по идентификаторам карт и категорий и только после этого отображается пользователю.

Задание: реализовать в существующем *Android*-приложении функцию *on-device LLM*, которая выполняет смысловую ревизию коллекции дисконтных карт и работает полностью автономно.

1. Подключить библиотеку *llamacpp-kotlin* и нативные библиотеки *llama.cpp* для 64-битных архитектур, не добавляя сетевых вызовов и внешних программных интерфейсов.

2. Реализовать интерфейс *LlmRepository* и его реализацию *LlmRepositoryImpl*, а также *UseCase*, который собирает данные коллекции, формирует промпт, вызывает модель и обрабатывает результат.

3. Обеспечить обработку состояний *Loading*, *Success* и *Error* через *StateFlow*, выделив отдельное состояние загрузки модели.

4. Добавить проверку совместимости устройства и корректное поведение при невозможности запуска модели.

5. Реализовать кэширование результата анализа в *Room* и сброс кэша при изменении коллекции карт.

6. Разработать пользовательский интерфейс выбора модели, запуска анализа, отображения прогресса, результата и ошибок.

Ход работы

1. Анализ исходного проекта. Исходное приложение представляет собой менеджер дисконтных карт на *Kotlin* и *Jetpack Compose*. В нём уже используются *Koin* для внедрения зависимостей, *Room* для хранения карт, *Retrofit* для синхронизации с локальным сервером, *ML Kit* для сканирования штрихкодов и навигация *Compose*. Карта описывается сущностью *DiscountCard* с полями *id*, *shopName*, *barcodeNumber*, *categoryId*, *colorHex* и *isUserAdded*. Справочник категорий загружается из *JSON* и содержит категории Супермаркеты, Одежда, Кафе и рестораны, Аптеки.

В ходе анализа обнаружен дефект исходной конфигурации: файл *categories.json* находился внутри каталога исходного кода, а не в каталоге *assets*. Из-за этого *CategoryRepository* не находил справочник и возвращал пустой список, поэтому категории не отображались в интерфейсе. Файл перенесён в *app/src/main/assets/categories.json*. После переноса категории появились на главном экране, а *AI*-модуль получил корректный справочник для формирования промпта.

2. Подключение библиотеки *llamacpp-kotlin.* В файл *app/build.gradle.kts* добавлена зависимость *io.github.ljcamargo*:*llamacpp-kotlin*:0.4.0. Библиотека содержит нативные библиотеки только для *arm64-v8a* и *x86_64*, поэтому в *defaultConfig* добавлено ограничение *abiFilters.* Такое ограничение исключает установку приложения на 32-битные устройства, для которых нативные библиотеки отсутствуют, и делает поведение предсказуемым. Минимальная версия *Android* оставлена на уровне 24.

Библиотека предоставляет класс *LlamaHelper*, который инкапсулирует загрузку *GGUF*-файла, создание контекста и генерацию токенов. Взаимодействие построено на потоке событий *LLMEvent* с вариантами *Loaded*, *Started, Ongoing*, *Done* и *Error.* Для выбранной модели используется контекст длиной 2048 токенов, количество потоков выбирается библиотекой автоматически. Передача файла выполняется через файловый дескриптор, поэтому поддерживается выбор модели через системный файловый диалог и сохранение разрешения на чтение.

3. Архитектура *AI*-модуля. Интерфейс *LlmRepository* содержит *StateFlow* состояния движка, методы проверки совместимости, выбора, проверки и загрузки модели, генерации, остановки и освобождения ресурсов. Такая абстракция позволяет тестировать сценарий анализа без запуска нативной библиотеки и заменять движок в модульных тестах.

Состояние движка описано *sealed*-интерфейсом *LlmState.* Вариант *Idle* соответствует отсутствию загруженной модели, *ModelLoading* описывает отдельное состояние загрузки модели, *ModelReady* означает готовность к генерации, *Generating* содержит частичный текст ответа, *Success* хранит полный ответ, *Error* содержит сообщение об ошибке. Разделение *ModelLoading* и *Generating* позволяет показывать пользователю разные этапы работы и не смешивать загрузку файла с вычислением ответа.

Реализация *LlmRepositoryImpl* создаёт *LlamaHelper*, подписывается на его события и преобразует их в единый *StateFlow.* Для защиты от одновременных запросов используется *Mutex.* Генерация ограничена таймаутом 10 минут, поддерживается остановка по требованию пользователя и освобождение контекста. Флаг *modelLoaded* позволяет отличить ошибку загрузки модели от ошибки генерации и корректно разрешать повторный запуск анализа.

Сценарий *AnalyzeCollectionUseCase* получает текущий список карт через *Flow.first*, проверяет непустые коллекции карт и категорий, вычисляет отпечаток коллекции, при необходимости читает кэш, формирует промпт, вызывает *LlmRepository.generate*, передаёт ответ в *LlmResponseParser* и сохраняет результат в *Room.* Все операции выполняются в корутинах, поэтому не блокируют главный поток.

ViewModel AiReviewViewModel хранит *AiReviewUiState* в *StateFlow.* В состоянии находятся совместимость устройства, состояние движка, выбранная модель, признак готовности модели, результат анализа, признак выполнения и сообщение об ошибке. Результат *AnalysisResult* содержит сам анализ, признак кэша и время создания. *ViewModel* подписывается на *StateFlow* репозитория,

автоматически загружает ранее выбранную модель, читает кэш при открытии экрана и предоставляет методы выбора модели, загрузки, анализа и остановки генерации.

Чтобы код было проще читать, модели, интерфейс и реализация репозитория объединены в файле *LlmRepository.kt*, а промпт, парсер и сценарий анализа - в файле *AnalyzeCollectionUseCase.kt.* Число файлов *AI*-модуля уменьшено с одиннадцати до пяти, при этом разделение ответственности между слоями сохранено.

Проверка совместимости встроена в *LlmRepositoryImpl.* Она проверяет версию *Android*, наличие 64-битной архитектуры *arm64-v8a* или *x86_64* и объём оперативной памяти. Минимальный порог составляет 3 ГБ: модель занимает около 1,1 ГБ, а оставшаяся память нужна контексту, нативным буферам и сборщику мусора. При несовместимости возвращается понятная причина, экран показывает предупреждение, а основная кнопка блокируется. Это и есть корректная деградация функции без падения приложения.

Выбор и проверка файла модели также входят в *LlmRepositoryImpl.* Сохранённый *URI* хранится в *SharedPreferences, GGUF*-файлы ищутся в *files/models* и *externalFilesDir/models*, проверяются сигнатура *GGUF* и минимальный размер файла. Если файл не выбран или не прошёл проверку, пользователь получает сообщение, а приложение остаётся работоспособным. Модель не включается в *APK* из-за размера и авторских прав, поэтому пользователь выбирает её самостоятельно.

LlmPromptBuilder в файле *AnalyzeCollectionUseCase.kt* формирует промпт в формате *ChatML.* Системная часть задаёт задачу проверки, требует использовать только предоставленные категории и карты и возвращать *JSON* по заданной схеме. Пользовательская часть содержит список категорий с идентификаторами и список карт с идентификатором, названием магазина, штрихкодом и текущей категорией. Ответ должен содержать *summary* и массив *recommendations* с полями *cardId, suggestedCategoryId, suggestedCategoryName* и *reason.* Название категории запрашивается дополнительно, чтобы уменьшить влияние ошибок в числовых идентификаторах. Если ошибок нет, массив остаётся пустым.

LlmResponseParser извлекает *JSON* из ответа, в том числе из *Markdown*-блока, и разбирает его через *Gson.* Затем проверяются идентификаторы: рекомендация отбрасывается, если карта или категория отсутствует в текущей коллекции, если предложена та же категория или если карта уже встречалась в ответе. Сначала парсер пытается найти категорию по названию *suggestedCategoryName*, затем по числовому *suggestedCategoryId.* Название из ответа сопоставляется со справочником, а в результат подставляется каноническое название категории. Если *JSON* повреждён, результат помечается как неструктурированный, а текст ответа сохраняется и отображается без списка рекомендаций.

Для кэширования используется сущность *AiAnalysisCache* с одной строкой *id*=1, *JSON*-результатом, отпечатком коллекции *cardsHash* и временем создания. Отпечаток вычисляется по идентификаторам, названиям магазинов

и категориям карт, а также по справочнику категорий и сворачивается в строку через *hashCode*. При изменении любой карты или категории отпечаток меняется, поэтому старый результат не показывается. База данных обновлена до версии 3, добавлена миграция *MIGRATION_2_3*, создающая таблицу *ai_analysis_cache*.

4. Загрузка модели и проверка совместимости. В тестовой конфигурации модель *qwen2.5-1.5b-instruct-q4_k_m.gguf* размером 1 117 320 736 байт размещена в каталоге *externalFilesDir/models*. При открытии экрана *LlmRepositoryImpl* находит файл, проверяет сигнатуру *GGUF*, после чего *ViewModel* запускает загрузку. На эмуляторе загрузка заняла около 20 секунд, в интерфейсе отображались отдельное состояние *ModelLoading* и индикатор прогресса.

Проверка совместимости на эмуляторе вернула положительный результат: архитектура *x86_64*, образ *android-37.1*, 4 ГБ оперативной памяти. При несовместимости устройство получает сообщение с причиной, например об отсутствии 64-битной архитектуры или недостаточном объёме ОЗУ. В этом случае *AI*-функция не запускается, но остальные возможности приложения продолжают работать.

5. Формирование промпта. Промпт собирается из данных, которые уже есть в приложении, поэтому не требуется передавать коллекцию во внешний сервис. Все карты перечисляются с текущей категорией, что позволяет модели сравнить смысл магазина с названием категории. Инструкция требует объяснять причину перемещения, а не только указывать новую категорию. Корректность промпта проверяется модульным тестом: тест контролирует наличие маркеров *ChatML*, всех карт, всех категорий и схемы *JSON*.

6. Генерация и состояния. При запуске анализа *ViewModel* вызывает *UseCase*, а тот обращается к *LlmRepository.generate*. Репозиторий переводит состояние в *Generating*, получает события *Ongoing* и обновляет частичный текст, затем по событию *Done* возвращает полный ответ. В интерфейсе отображаются индикатор, частичный текст и кнопка остановки. По завершении состояние меняется на *Success*, а после разбора ответа *ViewModel* показывает готовый результат. При ошибке состояние *Error* содержит сообщение, а кнопка повторного запуска остаётся доступной, если модель уже загружена.

7. Постобработка и кэширование. После генерации ответ модели не отображается напрямую: он проходит проверку структуры и идентификаторов. Это важно, потому что небольшая модель может добавить пояснение вокруг *JSON*, перепутать идентификатор или предложить категорию, которой нет в справочнике. После проверки результат сохраняется в *Room*. При повторном открытии экрана *ViewModel* читает кэш, сравнивает отпечаток коллекции и показывает сохранённый результат с отметкой Из кэша. Если коллекция изменилась, кэш игнорируется и выполняется новый анализ.

8. Пользовательский интерфейс. На главном экране в верхней панели добавлена кнопка *AI*-проверка. Она открывает отдельный экран, содержащий

строку о локальной работе, имя выбранного *GGUF*-файла, одну основную кнопку, строку состояния, индикатор выполнения и простой текстовый результат. Основная кнопка меняет действие в зависимости от состояния: выбрать модель, загрузить модель, остановить генерацию или проверить карты. Выбор файла выполняется через *ActivityResultContracts.OpenDocument*, после чего приложение сохраняет разрешение на чтение, проверяет файл и автоматически загружает модель. Отдельных карточек, иконок и декоративных блоков нет, поэтому экран остаётся компактным. Ошибки выводятся красным текстом под кнопкой.

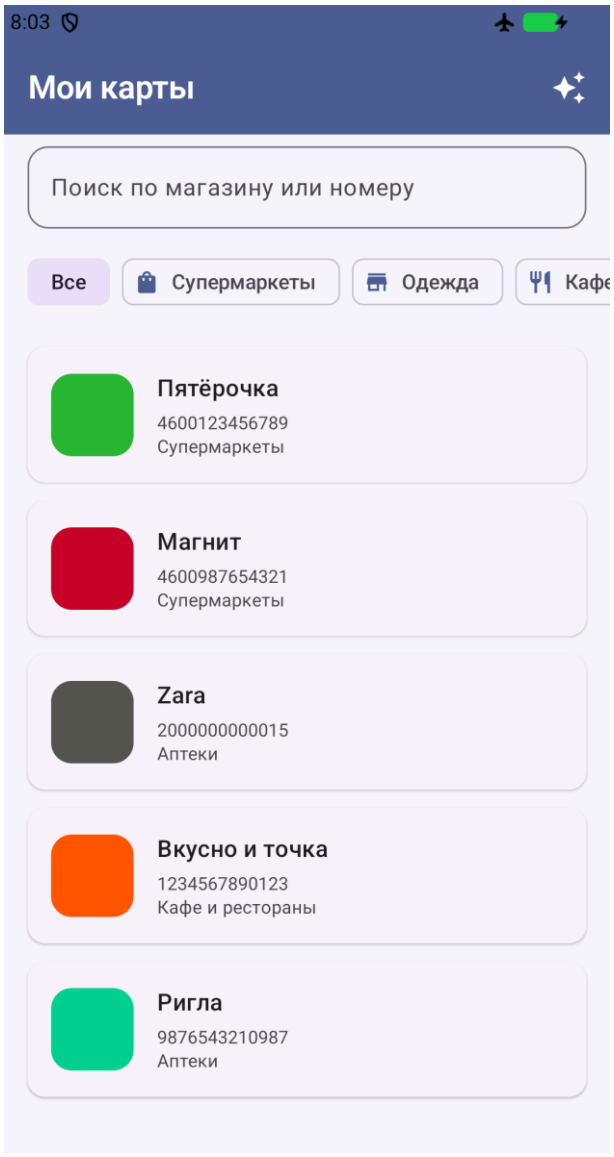


Рисунок 1 - Главный экран приложения с категориями карт и кнопкой *AI*-проверки

На рисунке 1 видно, что после исправления пути к *categories.json* на главном экране отображаются категории Супермаркеты, Одежда, Кафе и рестораны, Аптеки. В правой части верхней панели расположена кнопка *AI*-проверка, через которую открывается новый экран. Для проверки сценария рекомендации карта *Zara* временно перенесена в категорию Аптеки: на рисунке она показана именно в этом состоянии.

9. Тестирование. Сборка проекта выполнена командой *assembleDebug*, сборка завершилась успешно. Модульные тесты запущены командой *testDebugUnitTest*: выполнено 7 тестов, ошибок и пропусков нет. Тест *LlmPromptBuilderTest* проверяет структуру *ChatML* и наличие всех карт и категорий в промпте. Четыре теста *LlmResponseParserTest* проверяют разбор *JSON* внутри *Markdown*-блока, приоритет названия категории над числовым идентификатором, отбрасывание неизвестных карт и совпадающих категорий, а также корректный переход к неструктурированному результату при повреждённом *JSON*. Два теста *AnalyzeCollectionUseCaseTest* проверяют полный сценарий с разбором ответа, сохранением в кэш и повторным чтением из кэша, а также инвалидацию кэша после изменения категории карты.

Функциональная проверка выполнена на эмуляторе *Medium_Phone* с архитектурой *x86_64*, образом *android-37.1* и 4 ГБ оперативной памяти. Модель *qwen2.5-1.5b-instruct-q4_k_m.gguf* размером около 1,04 ГиБ была заранее скопирована в каталог *externalFilesDir/models* и автоматически найдена приложением. Для подтверждения автономности на эмуляторе включён авиарежим. Загрузка модели заняла около 20 секунд, генерация на эмуляторе заняла от одной до трёх минут в зависимости от длины ответа, что связано с ограниченной производительностью центрального процессора.

Для проверки смысловой ревизии карта *Zara* была временно помещена в категорию Аптеки. Чтобы малая модель не отвлекалась на остальные записи, сценарий проверен на тестовой коллекции из одной карты *Zara*; на рисунке 1 показана полная коллекция с той же картой в категории Аптеки. Модель обнаружила несоответствие и предложила вернуть карту в категорию Одежда, объяснив решение тем, что *Zara* является магазином одежды, а не аптекой. Ответ был разобран парсером, проверен по идентификаторам и сохранён в *Room*. На рисунке 5 показан этот же результат при повторном открытии экрана: отметка Из кэша подтверждает, что данные прочитаны из локального кэша без повторной генерации.

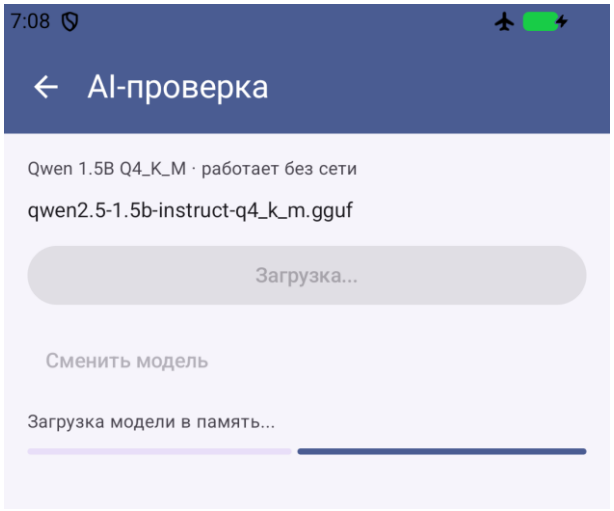


Рисунок 2 - Экран *AI*-проверки в состоянии загрузки модели

На рисунке 2 показано отдельное состояние *ModelLoading*. Пользователь видит имя файла, кнопку Загрузка... и строку Загрузка модели в

память. Основная кнопка в этот момент недоступна, что исключает обращение к незагруженной модели.

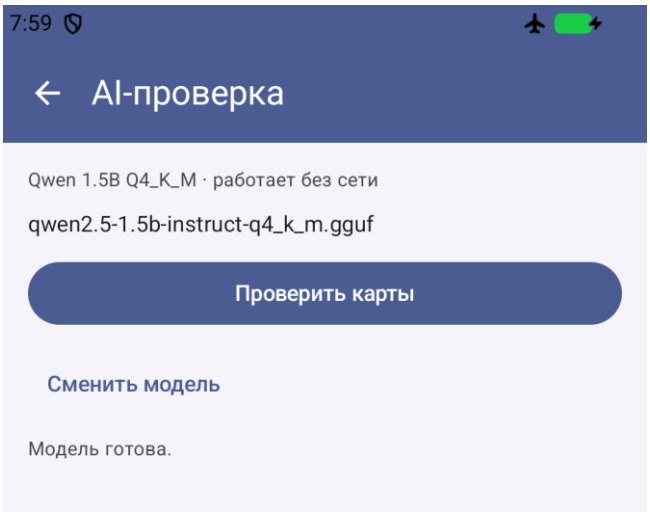


Рисунок 3 - Экран *AI*-проверки после загрузки модели

На рисунке 3 модель находится в состоянии *ModelReady*. Отображаются имя файла, кнопка Проверить карты и строка Модель готова. Отдельный блок совместимости не выводится: при несовместимости на его месте появляется причина, а основная кнопка блокируется.

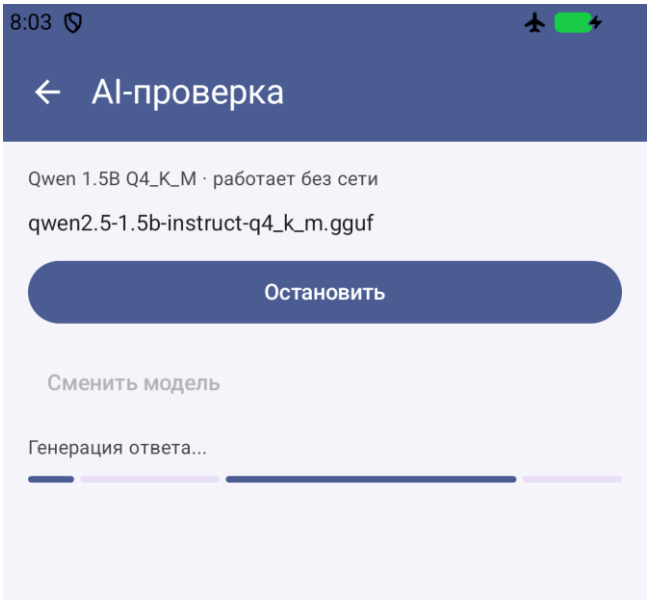


Рисунок 4 - Состояние генерации ответа при включённом авиарежиме

На рисунке 4 показан процесс генерации при включённом авиарежиме. В верхней части экрана виден значок самолёта, при этом модель продолжает работу, что подтверждает отсутствие сетевых вызовов. Во время генерации доступна кнопка остановки, а индикатор показывает, что вычисления продолжаются.

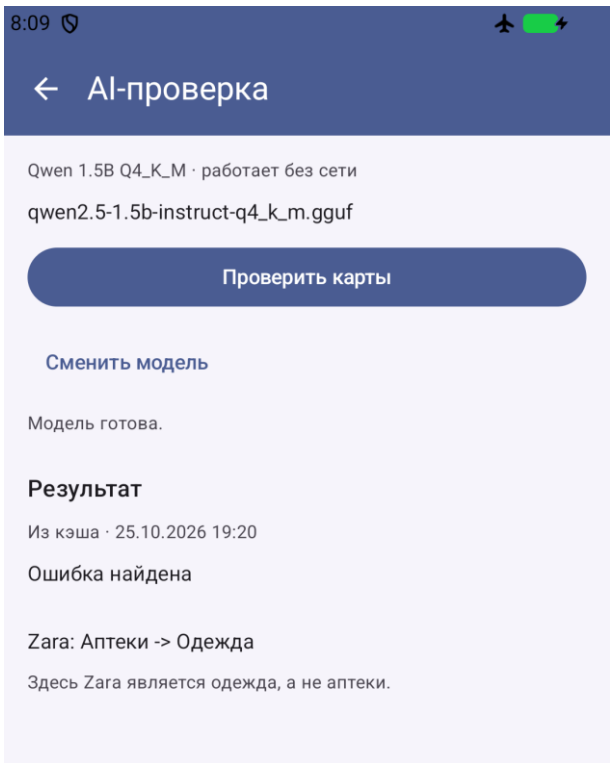


Рисунок 5 - Результат смысловой проверки, полученный из кэша *Room*

На рисунке 5 показан результат, прочитанный из кэша при повторном открытии экрана. Отметка Из кэша содержит дату и время сохранения. В результате приведена рекомендация *Zara*: Аптеки -> Одежда и причина, сформированная моделью. После перезапуска приложения модель загружается заново, но результат уже доступен пользователю без повторной генерации. Это уменьшает время ожидания и расход заряда аккумулятора.

Сценарий рекомендаций проверен не только модульным тестом, но и функционально на эмуляторе. Парсер получил *JSON* с картой, для которой предложена другая категория, проверил идентификаторы, подставил названия категорий из справочника и сформировал объект *AiRecommendation* с полями *cardId, shopName, currentCategoryName, suggestedCategoryName* и *reason*. Этот же объект отображается в интерфейсе в виде строки с названием магазина, текущей категорией, предлагаемой категорией и обоснованием.

Предусмотрены и обрабатываются следующие ошибочные ситуации: модель не выбрана, файл не является *GGUF*, размер файла слишком мал, устройство не поддерживает 64-битную архитектуру, объём ОЗУ меньше 3 ГБ, загрузка модели завершилась ошибкой, генерация превысила таймаут, ответ модели повреждён. В каждом случае приложение не завершается аварийно, а показывает сообщение и, где это возможно, предлагает повторить действие.

Вывод

В ходе лабораторной работы в *Android*-приложение добавлена функция *on-device LLM* для смысловой ревизии коллекции дисконтных карт. Подключена библиотека *llamacpp-kotlin*, реализованы *LlmRepository*, *LlmRepositoryImpl*, *AnalyzeCollectionUseCase*, *ViewModel* с *StateFlow* и отдельным состоянием загрузки модели, проверка совместимости, выбор

GGUF-файла, формирование промпта *ChatML*, постобработка *JSON* и кэширование результата в *Room*. Функция работает без сети и внешних программных интерфейсов, что подтверждено запуском в авиарежиме. На тестовой коллекции модель обнаружила карту *Zara* в категории Аптеки и предложила перенести её в категорию Одежда с объяснением причины. Сборка и 7 модульных тестов завершились успешно. Основные ограничения связаны с размером модели, требованиями к 64-битной архитектуре и объёму оперативной памяти, а также со скоростью генерации на мобильном процессоре.

Приложение *А*
(обязательное)
Листинг программы

```
sealed interface LlmState {
    data object Idle : LlmState
    data object ModelLoading : LlmState
    data object ModelReady : LlmState
    data class Generating(val partialText: String) : LlmState
    data class Success(val rawResponse: String) : LlmState
    data class Error(val message: String) : LlmState
}

class AnalyzeCollectionUseCase(
    private val cardRepository: CardRepository,
    private val categoryProvider: CategoryProvider,
    private val llmRepository: LlmRepository,
    private val cacheDao: AiAnalysisDao
) {
    suspend fun execute(forceRefresh: Boolean = false): AnalysisResult {
        val cards = cardRepository.cards.first()
        require(cards.isNotEmpty()) {
            "В коллекции нет карт для анализа."
        }
        val categories = categoryProvider.categories
        require(categories.isNotEmpty()) {
            "Не удалось загрузить категории карт."
        }
        if (!forceRefresh) {
            getCached()?.let { return it }
        }
        val prompt = LlmPromptBuilder.build(cards, categories)
        val rawResponse = llmRepository.generate(prompt)
        val analysis = parser.parse(rawResponse, cards, categories)
        val createdAt = System.currentTimeMillis()
        cacheDao.save(
            AiAnalysisCache(
                resultJson = gson.toJson(analysis),
                cardsHash = fingerprint(cards, categories),
                createdAt = createdAt
            )
        )
        return AnalysisResult(analysis, false, createdAt)
    }
}
```

```
private data class RecommendationDto(
    val cardId: Int? = null,
    val suggestedCategoryId: Int? = null,
    val suggestedCategoryName: String? = null,
    val reason: String? = null
)

val suggestedCategory = item.suggestedCategoryName
    ?.let { name -> categoriesById.values.firstOrNull {
        it.name.equals(name, ignoreCase = true)
    } }
    ?: item.suggestedCategoryId?.let(categoriesById::get)
    ?: return@mapNotNull null

@Entity(tableName = "ai_analysis_cache")
data class AiAnalysisCache(
    @PrimaryKey val id: Int = 1,
    val resultJson: String,
    val cardsHash: String,
    val createdAt: Long
)

val has64BitAbi = Build.SUPPORTED_ABIS.any {
    it == "arm64-v8a" || it == "x86_64"
}
if (!has64BitAbi) {
    return DeviceCompatibility(false, "Нет 64-битной архитектуры.")
}
```